ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
SCUOLA DI INGEGNERIA E ARCHITETTURA (SEDE DI BOLOGNA)
Anno Accademico 2024/2025

RELAZIONE DI FINE TIROCINIO CURRICULARE

svolto dallo studente:

Tommaso Magelli ( )

iscritto al Corso di Studio in

Automation Engineering (9217)

presso:

Agri-tech startup

sul seguente argomento:

Research and Development on trajectory planning algorithms for autonomous vehicles with trailers – In-Row Navigation

Studente:
Tommaso Magelli

_____
(firma)

Tutor Accademico:
Prof. Riccardo Fini

_____
(firma per approvazione della relazione finale)

Referente Struttura Ospitante:
Simone Rossi

_____
(firma per approvazione della relazione finale)

Table of Contents

1. **Introduction**
    - 1.1. Context and Motivation
    - 1.2. Internship Objectives and Scope
    - 1.3. Report Structure

2. **Problem Analysis: The State of the Art in Tractor-Trailer Navigation**
    - 2.1. The Tractor-Trailer System: A Complex Articulated Vehicle
    - 2.2. Key Kinematic and Dynamic Challenges
        - 2.2.1. The Off-Tracking Phenomenon
        - 2.2.2. Non-Holonomic Constraints
        - 2.2.3. Reverse Motion Instability: The Jackknifing Problem

3. **Core Methodologies in Autonomous Navigation**
    - 3.1. The Perception-Planning-Control Pipeline
    - 3.2. World Representation and Configuration Space
    - 3.3. Path Planning Algorithms
        - 3.3.1. Graph-Search Planners: Dijkstra's Algorithm
        - 3.3.2. Sampling-Based Planners: The Probabilistic Roadmap (PRM)
    - 3.4. Path Tracking Control
        - 3.4.1. The Pure Pursuit Controller
    - 3.5. Online vs. Offline Path Planning
    - 3.6. Path Primitives: Dubins & Reeds-Shepp Paths
    - 3.7. Localization and State Estimation

4. **Literature Review**
    - 4.1. A Modular Approach Based on Path Modification (Recker et al., 2021)
    - 4.2. Strategies for Reverse Motion and Open-Field Planning (Moreno, 2021)
    - 4.3. Optimization and MPC-Based Approaches

5. **Application Development and Implementation**
    - 5.1. System Architecture and Simulation Environment
    - 5.2. The Mission Decomposition Strategy
    - 5.3. Implementation of Path Segment Generation
        - 5.3.1. In-Row and Open-Field Navigation
        - 5.3.2. Headland Turning with Bézier Curves
    - 5.4. Vehicle Simulation and Control Loop
    - 5.5. A Deeper Look into Bézier Curve Control
    - 5.6. Choice of Parametric Curve: Bézier vs. Splines

6. **Results and Discussion**
    - 6.1. Mission Planning and Execution: The Snap-Point Logic
    - 6.2. Code Architecture and Modularity
    - 6.3. Performance of Headland Turn Generation
    - 6.4. Preliminary Development for Open-Field Navigation
    - 6.5. Discussion of Limitations and Future Work

7. **Conclusions**

8. **References**

1. Introduction

1.1. Context and Motivation

This internship focuses specifically on the autonomous navigation of tractor-trailer systems, a common vehicle configuration in modern agriculture used for tasks such as spraying, harvesting, and transportation.

Unlike a rigid vehicle, an articulated tractor-trailer system presents a unique set of challenges. Its motion is governed by complex kinematic and dynamic constraints, making path planning and control a non-trivial problem. The behavior of the trailer, especially during turns, must be accurately predicted and controlled to avoid collisions with crops, infrastructure, or other obstacles. This is particularly critical in high-value settings like vineyards and orchards, where rows are narrow and headland areas for turning are limited.

1.2. Internship Objectives and Scope

The primary objective of this internship was to conduct a thorough analysis of these challenges and, subsequently, to develop a software simulation environment in MATLAB capable of planning, simulating, and validating navigation missions for a tractor-trailer vehicle. The project aimed to create a functional environment to serve two purposes: first, to implement and test fundamental path planning and control algorithms, and second, to establish a robust code base for future research and development.

The scope of the work included a literature review of existing methodologies, the implementation of an easy kinematic model for the vehicle, the development of a mission planner, and the integration of a path tracking controller.

1.3. Report Structure

This report documents the activities and findings of the internship. Chapter 2 provides an analysis of the problem, detailing the key challenges in tractor-trailer navigation. Chapter 3 elaborates on the core algorithms and methodologies that form the foundation of autonomous navigation, with a focus on those implemented in this project. Chapter 4 presents a review of relevant scientific literature. Chapter 5 describes the architecture and functionality of the developed MATLAB application. Chapter 6 showcases the results obtained from the simulation environment and discusses them. Finally, Chapter 7 concludes the report, summarizing the work accomplished and outlining directions for future research.

2. Problem Analysis: The State of the Art in Tractor-Trailer Navigation

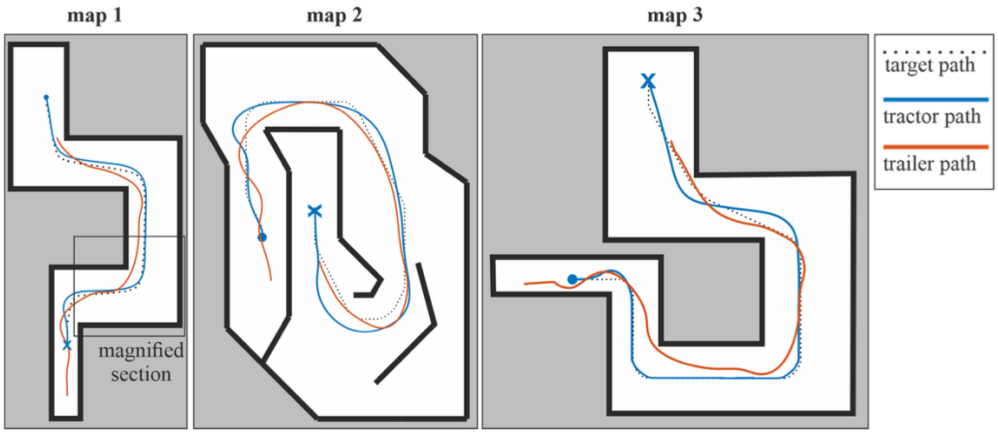
2.1. The Tractor-Trailer System: A Complex Articulated Vehicle

An articulated vehicle consists of two or more rigid bodies connected by joints. The system considered here is a standard tractor-trailer, where a powered tractor unit pulls a passive trailer via a pivot joint, often called a kingpin. This articulation grants the system high maneuverability but introduces significant complexity to its control.

2.2. Key Kinematic and Dynamic Challenges

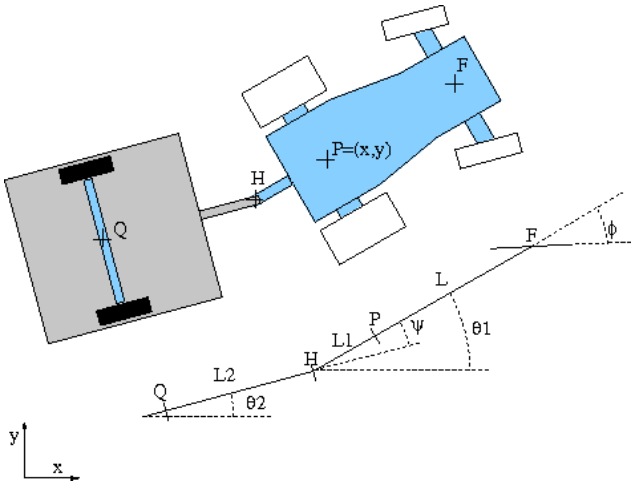
2.2.1. The Off-Tracking Phenomenon

The most significant kinematic challenge is off-tracking. This is the phenomenon where the trailer's wheels do not follow the same path as the tractor's wheels during a turn. The trailer tends to cut the corner, following a path with a smaller radius than the tractor. The magnitude of this off-tracking depends on the vehicle's geometry (wheelbase of tractor and trailer, distance from rear axle to hitch) and the sharpness of the turn. Accurate modeling of off-tracking is essential for collision avoidance, as a planner that only considers the tractor's path would risk the trailer colliding with obstacles on the inside of a turn.



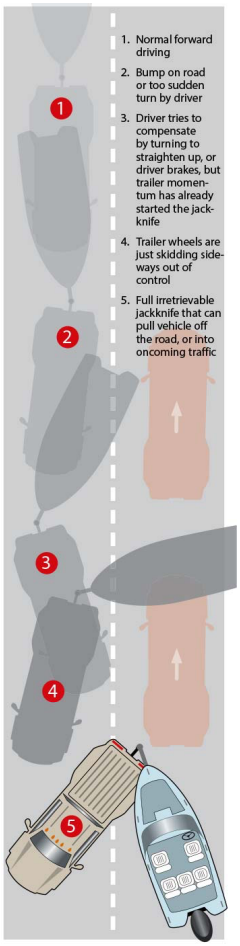
2.2.2. Non-Holonomic Constraints

Like most ground vehicles, the tractor-trailer system is subject to non-holonomic constraints. This means that its velocity is constrained; it can move forward and backward and turn, but it cannot move sideways instantaneously. This constraint dictates that any valid path for the vehicle must be continuous and have a continuous tangent, with a curvature that does not exceed the maximum allowed by the vehicle's steering mechanism. Path planning algorithms must generate trajectories that respect these constraints to be physically feasible.



2.2.3. Reverse Motion Instability: The Jackknifing Problem

While challenging in forward motion, controlling a tractor-trailer in reverse is notoriously difficult due to its instability. Any small error in steering during a reverse maneuver can be amplified, causing the angle between the tractor and trailer to increase uncontrollably until the trailer "folds" onto the tractor. This condition, known as jackknifing, is highly dangerous and can cause significant damage. It represents a major roadblock for full autonomy in environments requiring complex maneuvers, such as docking to a loading bay.





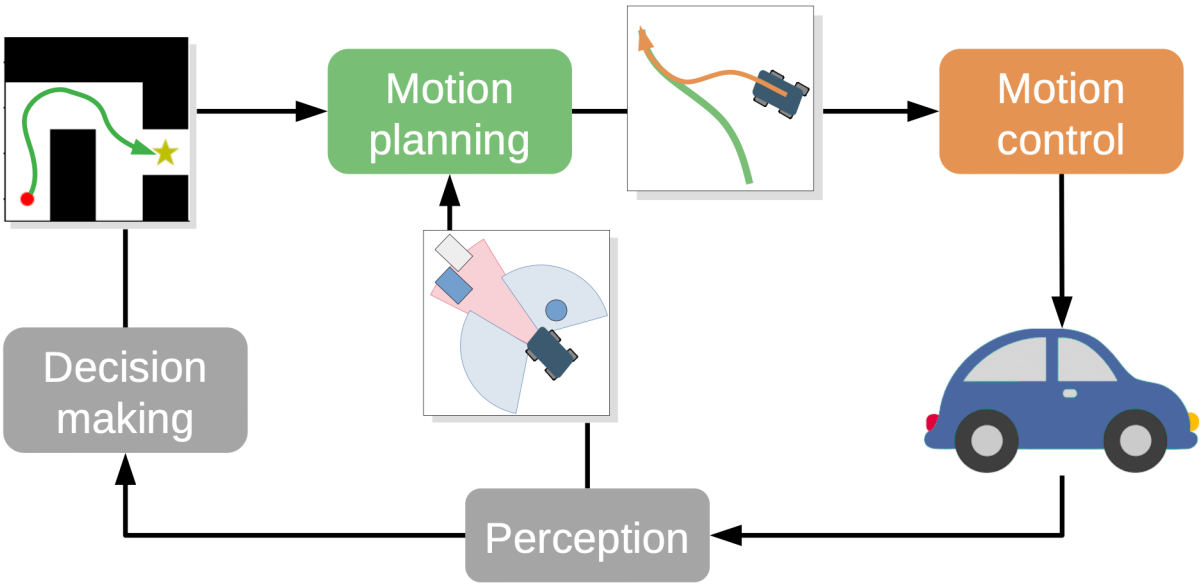
3. Core Methodologies in Autonomous Navigation

To address these challenges, a typical autonomous navigation system is structured as a pipeline of interconnected modules.

3.1. The Perception-Planning-Control Pipeline

The system continuously executes a loop:

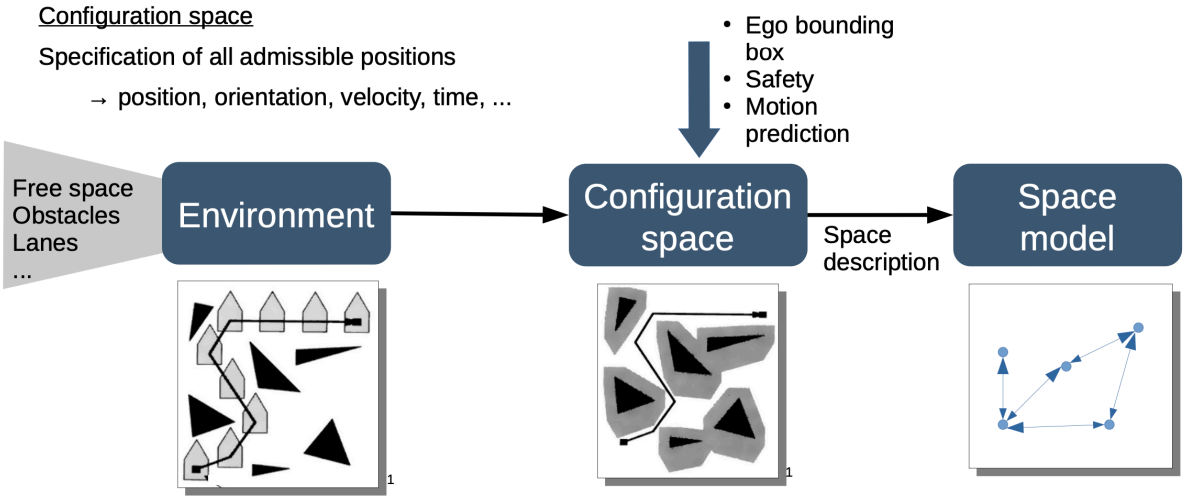
1. **Perception:** Sensors (like cameras, LiDAR, GPS) gather data about the environment and the vehicle's state.
2. **Decision Making / Planning:** This high-level module processes the perceived information to make strategic decisions (e.g., "turn at the next headland"). It then invokes a Motion Planner to generate a detailed, collision-free, and kinematically feasible trajectory.
3. **Control:** A low-level Motion Controller receives the planned trajectory and calculates the necessary actuator commands (e.g., steering angle, velocity) to make the vehicle follow the path as closely as possible.



This internship focused on the Planning and Control components within a simulated environment.

3.2. World Representation and Configuration Space

Before a path can be planned, the environment must be represented in a way the computer can understand. This is typically done using an occupancy grid map, where the world is divided into a grid of cells, each labeled as free, occupied, or unknown, 0,1 or x. To account for the robot's physical size, the problem is often transformed from the 2D world space into the Configuration Space (C-Space). In C-Space, the robot is represented as a single point, and obstacles are "inflated" by the robot's footprint. Planning a path for a point in C-Space is equivalent to planning a collision-free path for the full-sized robot in the real world.



3.3. Path Planning Algorithms

3.3.1. Graph-Search Planners: Dijkstra's Algorithm

One of the most fundamental classes of planning algorithms involves representing the free space as a graph and searching for the optimal path. A graph consists of nodes (representing locations) and edges (representing connections between locations), with each edge having an associated cost (e.g., distance).

Dijkstra's algorithm is a classic algorithm that finds the shortest path between a starting node and all other nodes in a weighted graph. It works by iteratively exploring the graph, always expanding the "closest" unvisited node.

PSEUDOCODE

```
function dijkstra(Graph, start_node):
  // Initialization
  for each node v in Graph:
    dist[v] := infinity  // Distance from start_node to v
    previous[v] := undefined  // Previous node in shortest path
  dist[start_node] := 0  // Distance from start_node to itself is 0
  Q := copy of all nodes in Graph // Priority queue of unvisited nodes

  // Main loop
  while Q is not empty:
    // Find the node u with the smallest distance in Q
    u := extract_min(Q)

    // For each neighbor v of u:
    for each neighbor v of u:
      alt := dist[u] + length(u, v)  // Possible shorter path via u
      if alt < dist[v]:
        dist[v] := alt  // Update shortest distance to v
        previous[v] := u // Update previous node in shortest path

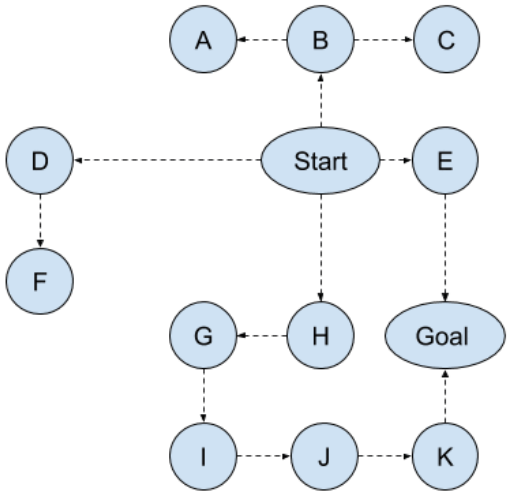
  return dist, previous // Return shortest distances and paths
```

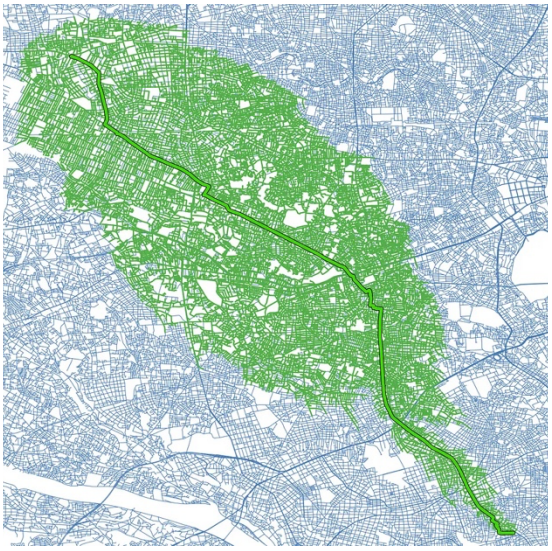
The process is as follows:

1. Initialize all nodes with an infinite distance cost, except for the start node, which has a cost of 0.
2. Maintain a set of unvisited nodes.
3. While the unvisited set is not empty, select the unvisited node with the lowest current cost (the "current" node).
4. For each neighbor of the current node, calculate the path cost by summing the current node's cost and the cost of the edge to the neighbor.
5. If this new path cost is less than the neighbor's previously recorded cost, update the neighbor's cost.
6. Mark the current node as visited and remove it from the unvisited set.

7. The algorithm terminates when the destination node is visited or all reachable nodes have been visited.

In this project, Dijkstra's algorithm was used to find the shortest path on a graph generated by the PRM planner.





3.3.2. Sampling-Based Planners: The Probabilistic Roadmap (PRM)

For complex, high-dimensional spaces, building a complete graph is infeasible. Sampling-based planners like the Probabilistic Roadmap (PRM) offer an efficient alternative. PRM works in two phases:

1. **Learning Phase:** Randomly sample a large number of configurations (nodes) in the C-Space, discarding any that are in collision. Then, try to connect nearby nodes with a straight-line path. If the path is collision-free, an edge is added to the graph (the roadmap).

PSEUDOCODE:

```
function PRM_BUILD(N, K):
 // Initialization
 V := empty_set      // Initialize an empty set of vertices
 E := empty_set      // Initialize an empty set of edges

 // Sample nodes
 for i = 1 to N do:
     q_rand := SAMPLE_FREE() // Generate a random configuration
     if q_rand is valid:   // Check for collision
         V := V union {q_rand}   // Add valid node to vertices
// Connect nodes
 for each q in V:
     N_q := K_NEAREST_NEIGHBORS(q, V, K) // Get K nearest neighbors
     for each q_prime in N_q:
         if LOCAL_PLANNER(q, q_prime) is collision-free:
             if (q, q_prime) is not in E:
                 E := E union {(q, q_prime)} // Add valid edge to graph

 return G = (V, E)       // Return the completed roadmap
```

2. **Query Phase:** Given a start and goal configuration, connect them to the nearest nodes in the roadmap. Then, use a graph-search algorithm like Dijkstra to find a path through the roadmap.

PSEUDOCODE:

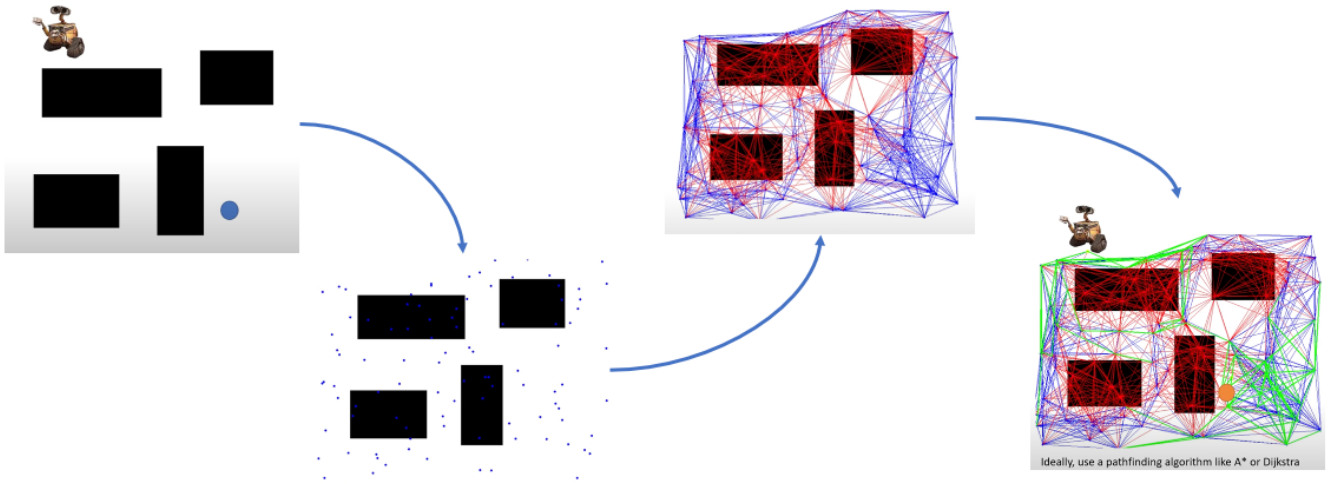
```
function PRM_QUERY(q_start, q_goal, G):
    // Connect start configuration to the roadmap
    N_start := K_NEAREST_NEIGHBORS(q_start, G.V, K)
    for each q_s in N_start:
        if LOCAL_PLANNER(q_start, q_s) is collision-free:
            G.E := G.E union {(q_start, q_s)}
    G.V := G.V union {q_start}

    // Connect goal configuration to the roadmap
    N_goal := K_NEAREST_NEIGHBORS(q_goal, G.V, K)
    for each q_g in N_goal:
        if LOCAL_PLANNER(q_goal, q_g) is collision-free:
            G.E := G.E union {(q_goal, q_g)}
    G.V := G.V union {q_goal}

    // Search for a path in the updated graph
    path := GRAPH_SEARCH(q_start, q_goal, G) // e.g., A* or Dijkstra's

    if path is found:
        return path
    else:
        return FAILURE
```

PRM is probabilistically complete, meaning that if a path exists, the probability of finding it approaches 1 as the number of samples increases.



3.4. Path Tracking Control

Once a geometric path is planned, a controller is needed to execute it.

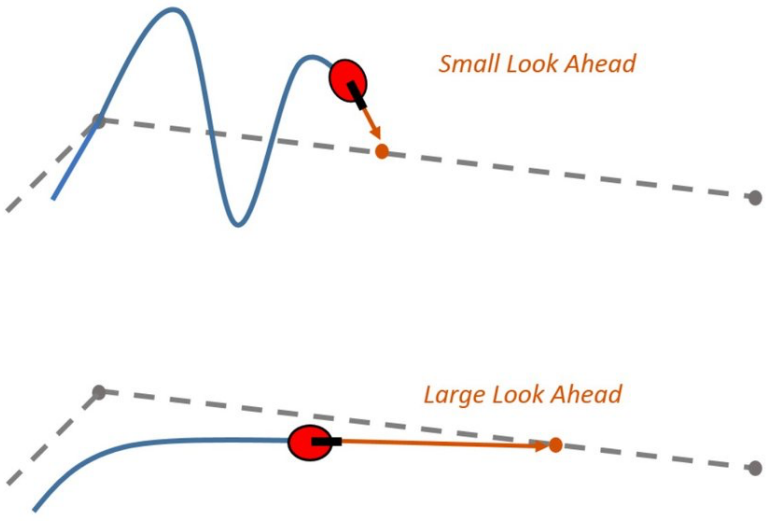
3.4.1. The Pure Pursuit Controller

The Pure Pursuit controller is a geometric path tracking controller commonly used for its simplicity and effectiveness. It does not require a complex dynamic model of the vehicle, relying instead on its kinematics.

The algorithm works as follows:

1. The controller identifies a target point on the reference path. This point is typically located a certain lookahead distance ahead of the vehicle's current position (e.g., the rear axle).
2. It calculates the curvature of a circular arc that connects the vehicle's current position to the target point.
3. It then computes the steering angle required to follow this arc.

The lookahead distance is a critical tuning parameter. A small lookahead distance results in more aggressive, responsive tracking but can lead to oscillations. A large lookahead distance produces smoother control but can cause the vehicle to cut corners more. In the developed application, this controller was implemented to drive the simulated tractor along the planned path segments.



3.5. Online vs. Offline Path Planning

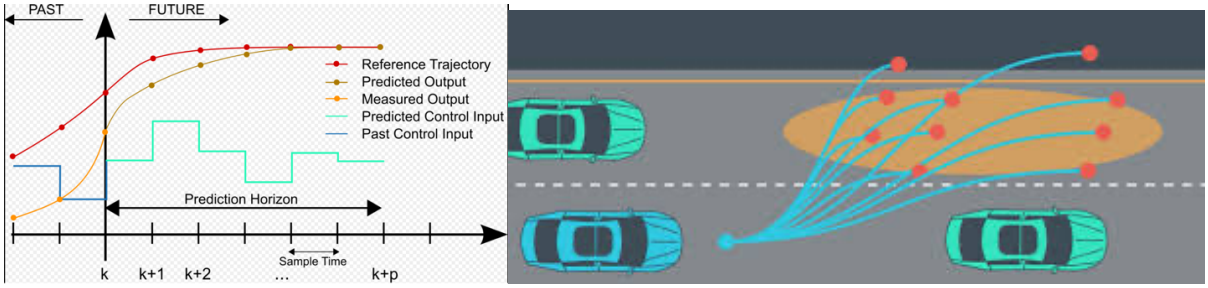
Path planning strategies can be broadly categorized into two paradigms: offline and online.

- **Offline Planning:** In this approach, a complete path from start to goal is computed *before* the vehicle begins to move. Planners like Dijkstra and PRM, when used to generate a full trajectory, fall into this category. They typically assume a static, known environment. The advantage is that the resulting path can be globally optimal (e.g.,

the shortest possible). The disadvantage is a lack of reactivity; if an unexpected obstacle appears, the entire path may need to be recomputed, which can be time-consuming.

- **Online Planning:** This paradigm, also known as local or reactive planning, involves computing or refining the path in real-time as the vehicle moves. This allows the system to react to dynamic changes in the environment. A prominent example of this approach is Model Predictive Control (MPC).

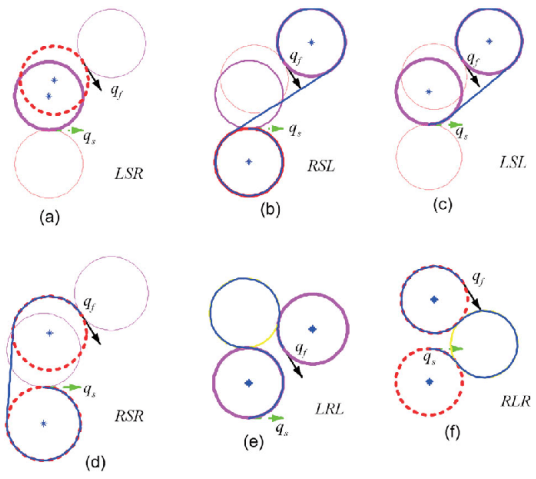
  MPC, also called Receding Horizon Control, is an advanced control method that uses a dynamic model of the system to predict its future evolution. At each time step, it solves an online optimization problem to find the optimal sequence of control actions (e.g., steering angles) over a finite time window called the prediction horizon. This optimization minimizes a cost function (e.g., deviation from a reference path, control effort) while satisfying system constraints (e.g., maximum steering angle). Only the first control action in the optimal sequence is applied. At the next time step, the process is repeated with new sensor measurements, effectively "receding" the horizon forward in time. This makes MPC inherently reactive and well-suited for handling complex, constrained systems like the tractor-trailer.



3.6. Path Primitives: Dubins & Reeds-Shepp Paths

For car-like robots with non-holonomic constraints, simply connecting two points with a straight line is often not a valid path. Specialized path primitives are needed to generate the shortest feasible path between two configurations (i.e., position and orientation).

- **Dubins Path:** A Dubins path finds the shortest path for a forward-only moving vehicle with a minimum turning radius. The optimal path is always composed of a sequence of at most three segments, which can be either a straight line (S) or a circular arc (C) at the maximum curvature. The possible combinations are of the type CSC (Curve-Straight-Curve) or CCC (Curve-Curve-Curve).
- **Reeds-Shepp Path:** Reeds-Shepp paths extend the Dubins path by allowing the vehicle to change direction (i.e., move in reverse). This introduces "cusps" into the path. While longer than Dubins paths, they are essential for maneuvers where reversing is necessary, such as parallel parking or navigating tight, cluttered spaces.

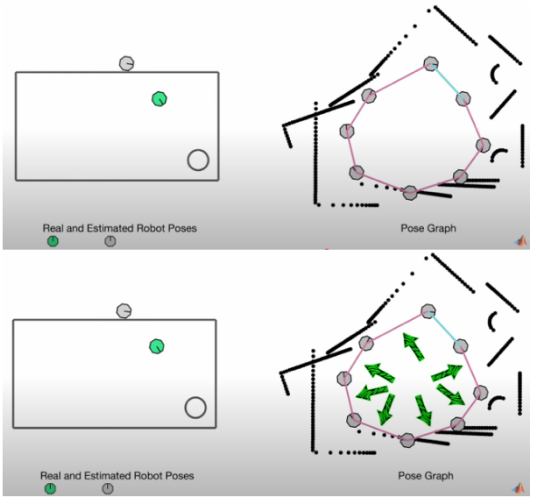


These primitives are fundamental building blocks in many motion planners, especially in sampling-based planners like RRT*, where they can be used to connect nodes in a kinematically feasible way.

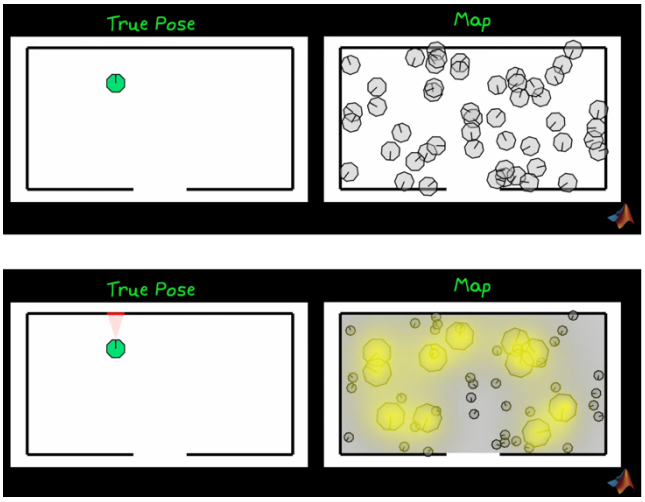
3.7. Localization and State Estimation

In a real-world application, the vehicle's exact position and orientation (its "pose") are never perfectly known. Odometry (tracking wheel rotations) is prone to drift, and GPS can be inaccurate or unavailable. Therefore, the robot must estimate its pose by fusing data from multiple noisy sensors. This problem is known as localization or state estimation.

- **Pose Graph Optimization:** This is a powerful technique often used in Simultaneous Localization and Mapping (SLAM). The robot's trajectory is represented as a pose graph, where each node is a vehicle pose at a certain point in time, and each edge represents a spatial constraint between poses (e.g., from odometry). When the robot recognizes a previously visited location (a "loop closure"), a new constraint is added to the graph. An optimization algorithm then adjusts all the poses in the graph simultaneously to minimize the error across all constraints. This corrects the entire trajectory at once, significantly reducing accumulated drift.



- **Particle Filters and Adaptive Monte Carlo Localization (AMCL):** These are probabilistic, filter-based methods for tracking a robot's pose. Instead of estimating a single pose, a particle filter represents the robot's belief about its position using a large set of weighted samples, or "particles." Each particle represents a possible pose. The set of particles is propagated over time using a motion model (which accounts for odometry noise). When new sensor measurements arrive (e.g., from a LiDAR scan), each particle's weight is updated based on how well its hypothetical pose explains the measurement. Particles that are inconsistent with sensor data are assigned low weights and eventually eliminated, while particles that are consistent are duplicated. This allows the filter to converge on the true pose of the vehicle and track it over time.



4. Literature Review

The development phase was preceded by a review of key scientific papers that address the tractor-trailer navigation problem.

4.1. A Modular Approach Based on Path Modification (Recker et al., 2021)

This paper proposes an efficient and modular approach that avoids complex global replanning. The core idea is to start with a pre-existing, possibly suboptimal path (e.g., from a simple planner that ignores some constraints). The algorithm then simulates the tractor-trailer's movement along this path to identify critical sections, primarily sharp curves, where collisions are likely. These unsafe sections are then locally replaced by dynamically generated, safe Bézier curves. The control points of the replacing Bézier curve are placed heuristically to push the path away from obstacles, ensuring the trailer has sufficient clearance. This method of local modification was a major inspiration for the headland turning strategy implemented in this project.

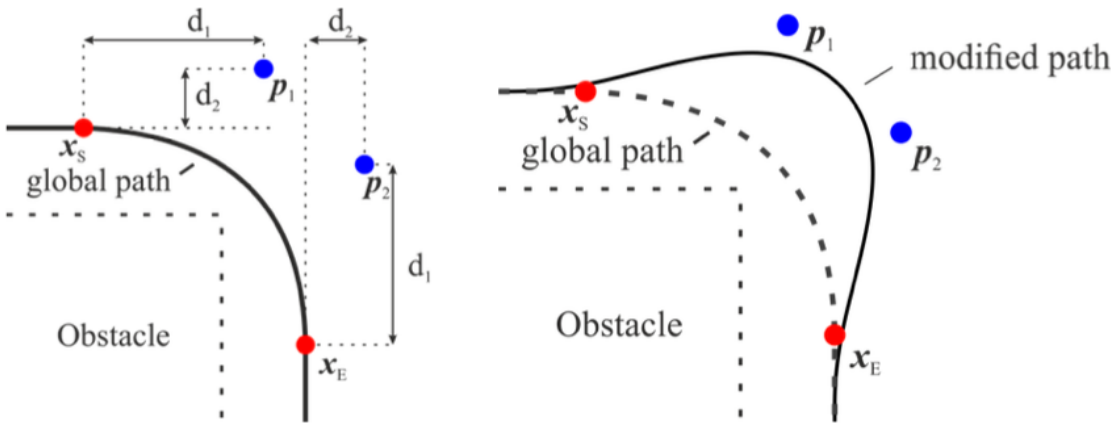


Figure 6: Bezier Curve with all control points (left: original; right: modified)

4.2. Strategies for Reverse Motion and Open-Field Planning (Moreno, 2021)

This thesis explores a comprehensive set of strategies for articulated vehicle navigation. A key contribution is a low-level controller designed to stabilize the hitch angle during reverse motion to prevent jackknifing. For forward motion in open fields, it combines a sampling-based planner (PRM) to navigate complex obstacle fields with kinematically constrained path primitives (Dubins curves) to ensure the generated paths are smooth and followable. This highlights the power of hybrid approaches that combine the strengths of different algorithms.

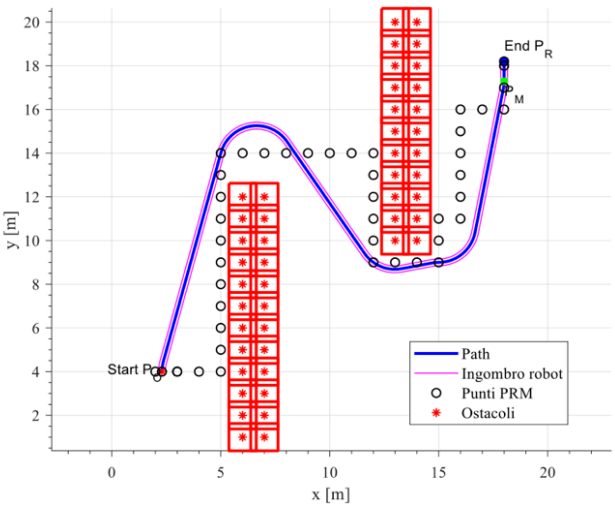
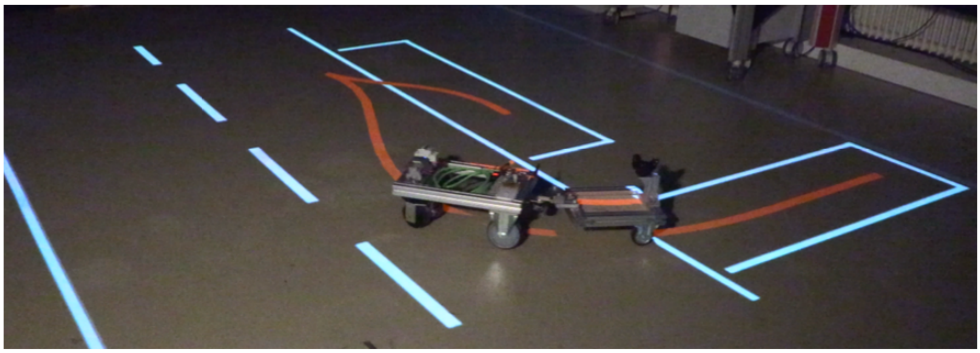


Figura 7.12 – Percorso ottenuto utilizzando la logica di interpolazione descritta

4.3. Optimization and MPC-Based Approaches

More advanced approaches in the literature formulate the path planning problem as a numerical optimization problem. Model Predictive Control (MPC) is a prominent example. An MPC-based planner predicts the future evolution of the system over a short time horizon and solves an optimization problem at each time step to find the best control sequence that minimizes a cost function (e.g., deviation from a reference line) while satisfying all vehicle and environmental constraints. These methods are powerful and can handle complex dynamics explicitly but are computationally intensive.



5. Application Development and Implementation

5.1. System Architecture and Simulation Environment

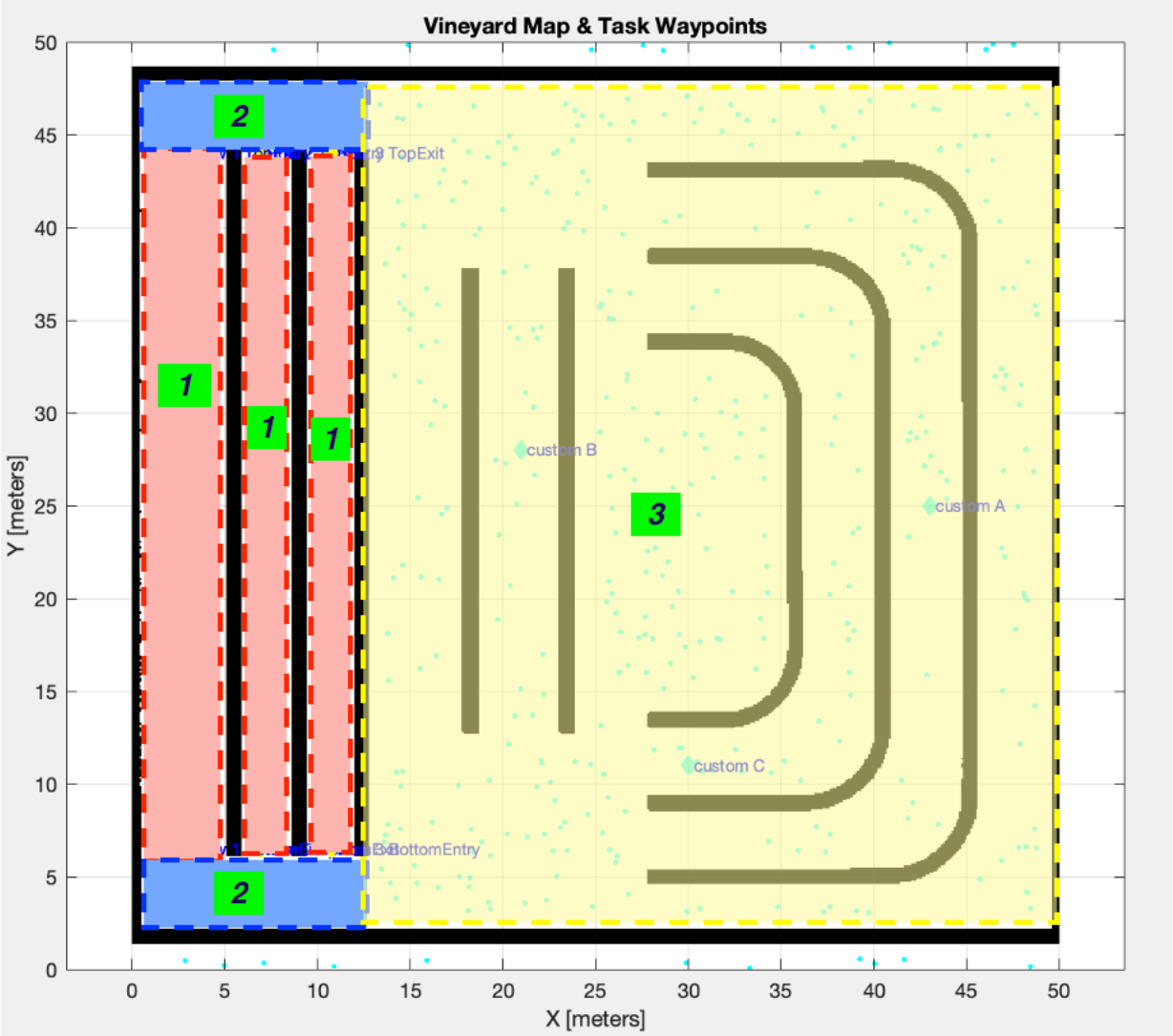
The final product of the internship is a software application developed entirely in MATLAB. The environment allows a user to:

- Load a binary occupancy map representing a field with rows of crops (e.g., a vineyard).
- Define a high-level mission as a sequence of waypoints.
- Invoke the path planner to generate a complete, multi-segment trajectory.
- Visualize the planned path, the environment, and an animation of the tractor-trailer system executing the mission.

5.2. The Mission Decomposition Strategy

A key architectural decision was to simplify the planning problem by decomposing a full mission into a sequence of distinct segments. This avoids the complexity of solving a single, large-scale planning problem. Based on the waypoints, each segment is classified into one of three types:

1. **In-Line Navigation:** A straight-line path for traveling up or down a crop row.
2. **End-of-Row Turning:** A complex maneuver to transition from the exit of one row to the entrance of another.
3. **Open-Field Navigation:** A path connecting two arbitrary points in an unstructured area.



This modular approach allows for the use of specialized, efficient algorithms for each part of the mission.

5.3. Implementation of Path Segment Generation

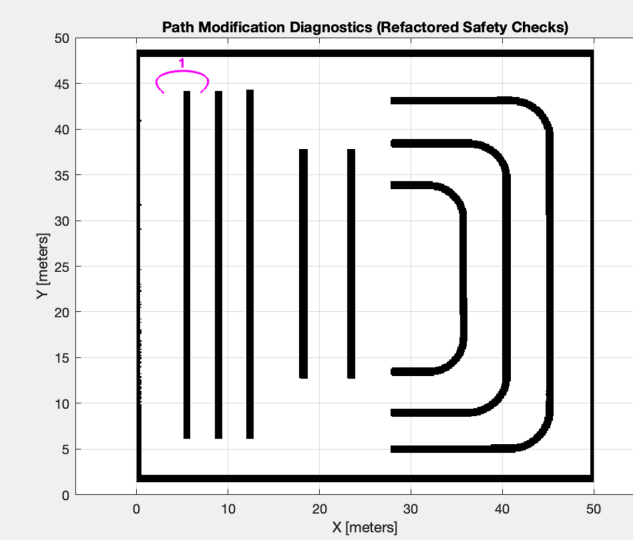
5.3.1. In-Row and Open-Field Navigation

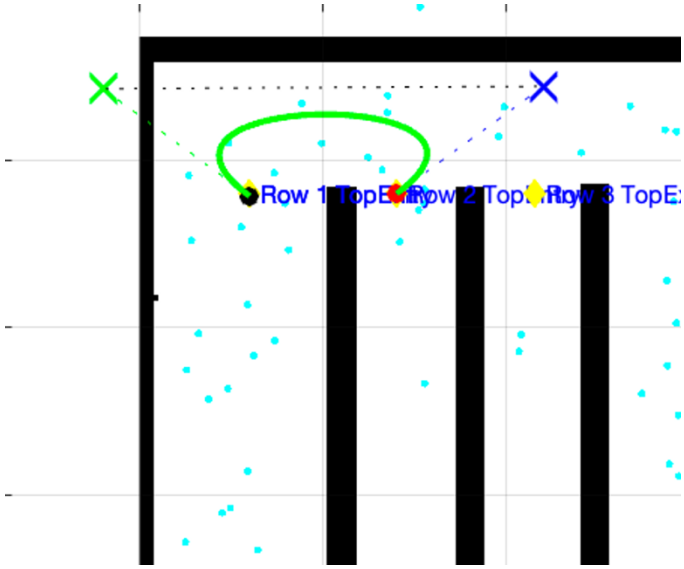
For in-line navigation, the path is simply a straight line connecting the start and end waypoints. For the open-field navigation task, the implemented solution uses the PRM algorithm to build a roadmap of the free space, followed by Dijkstra's algorithm to find the shortest path on the map. While functional, this part of the application was considered out of the main scope and is an area for future development.

5.3.2. Headland Turning with Bézier Curves

The core of the planner's intelligence lies in its handling of end-of-row turning. Inspired by the literature, these turns are generated using cubic Bézier curves. A Bézier curve is a parametric curve defined by a set of control points. Instead of a complex optimization, the planner uses a fixed-parameter heuristic to place the four control points (P0, P1, P2, P3). P0 and P3 are the start and end points of the turn (exit of one row, entrance of the next).

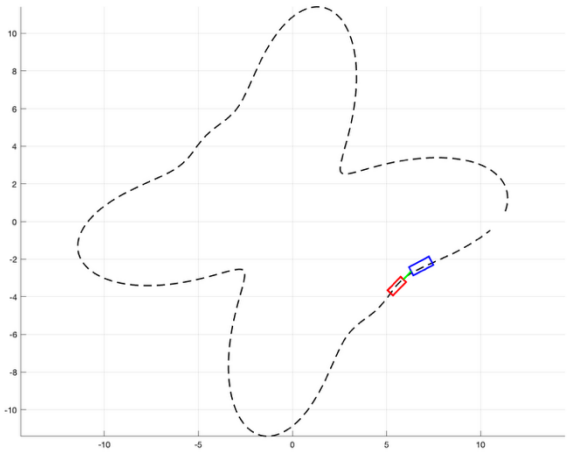
The intermediate control points, P1 and P2, are positioned based on the vehicle's exit/entry headings and fixed offset distances. This heuristic is designed to generate a wide, smooth "U" turn that provides ample space for the trailer, preventing it from cutting the corner and colliding with the crop rows.





5.4. Vehicle Simulation and Control Loop

To validate the planned paths, a kinematic simulator was developed. The simulator takes the generated path as a reference. The Pure Pursuit controller calculates the required steering angle for the tractor at each time step. The position and orientation of the trailer are then updated based on the kinematic model that links its motion to the tractor's rear axle via the hitch point. This allows for a realistic simulation of the off-tracking phenomenon.



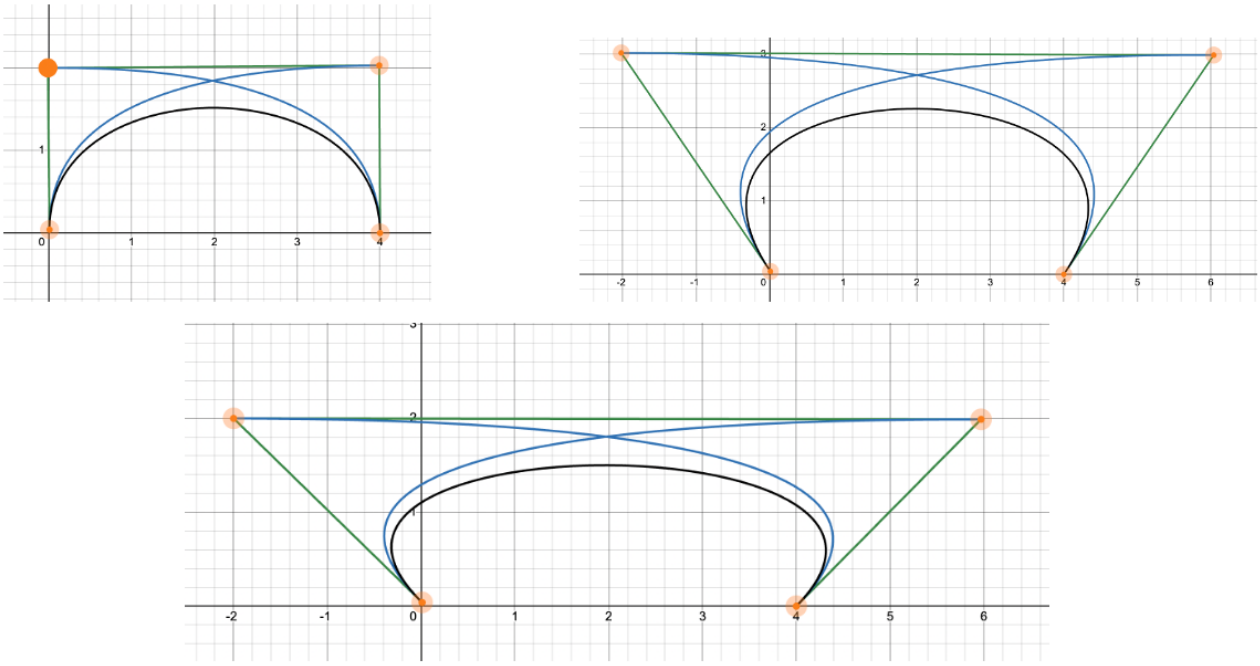
5.5. A Deeper Look into Bézier Curve Control

A cubic Bézier curve is a parametric curve defined by four points: a start point (P0), an end point (P3), and two intermediate control points (P1 and P2). The curve starts at P0 and ends at P3. The control points P1 and P2 act as "magnets," pulling the curve towards them without necessarily touching them. The line segment from P0 to P1 defines the tangent direction of the curve at its start, and the line from P2 to P3 defines its tangent at the end. The length of these tangent segments influences the "fullness" or "velocity" of the curve.

In this project, instead of using a complex optimization algorithm to find the ideal curve, a geometric heuristic was developed to place the control points P1 and P2 for headland turns. This heuristic is based on the geometry of the vineyard:

- **Longitudinal Placement:** The control points are pushed "forward" into the headland along the tangent of the start (P0) and end (P3) points. This distance is set empirically to roughly three-quarters of the headland's length. This ensures the curve extends deep into the open space, creating a wide arc rather than a sharp turn.
- **Lateral Placement:** This is the key to creating the desired "U" shape. The control points are also shifted laterally (perpendicular to the direction of travel).
    - The first control point, P1, is shifted laterally towards the center of the row adjacent to the one the vehicle is exiting.
    - Similarly, the second control point, P2, is shifted laterally towards the center of the row adjacent to the one the vehicle is entering.

This combined longitudinal and lateral placement forces the Bézier curve to bulge outwards, creating a wide, sweeping turn that naturally accommodates the trailer's off-tracking. The widest part of the turn occurs in the most open area, far from the crop rows, ensuring a safe and smooth maneuver.



5.6. Choice of Parametric Curve: Bézier vs. Splines

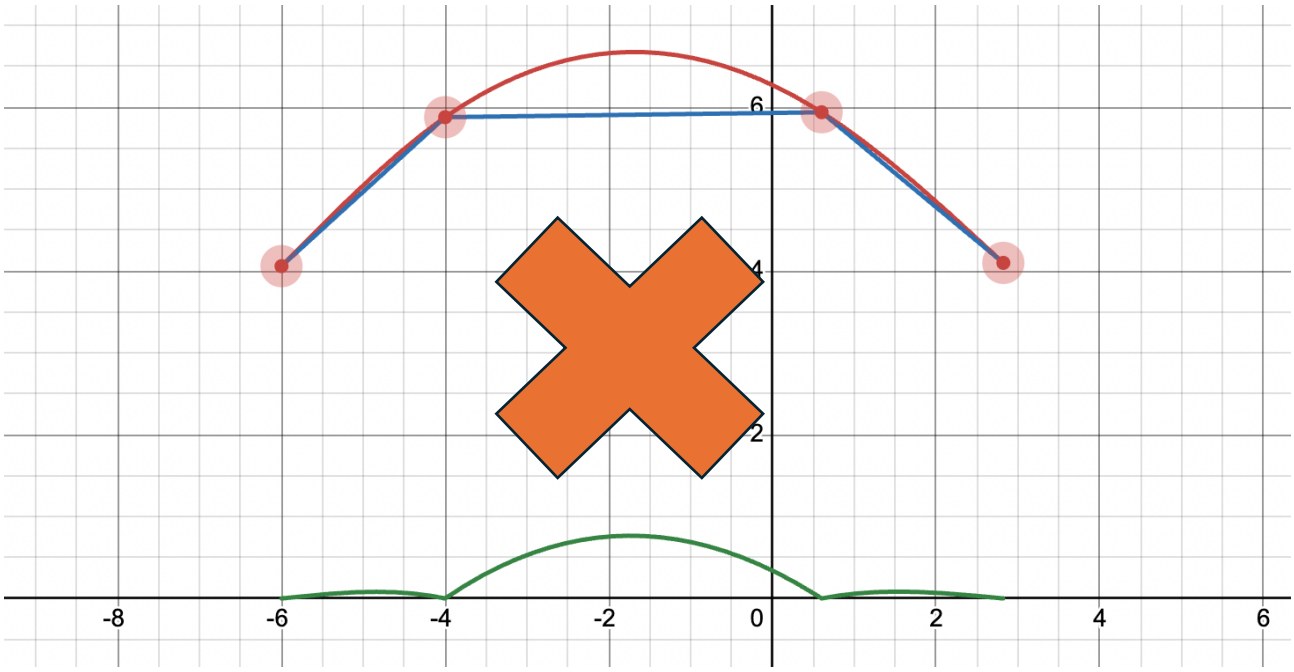
For generating the smooth turning paths, Bézier curves were chosen over other parametric alternatives, such as splines (e.g., B-Splines or NURBS), for several key reasons that are advantageous for this specific application.

- **Intuitive, Global Control:** A Bézier curve's shape is globally influenced by all of its control points. Moving a single control point alters the entire curve. While this can be a disadvantage for complex designs, it is ideal for generating a single, holistic maneuver like a headland turn. The four control points provide a simple and intuitive "handle" to define the start, end, and overall shape of the turn in a predictable way.
- **The Convex Hull Property:** This is a crucial feature for path planning. A Bézier curve is always contained entirely within the convex hull of its control points (the smallest convex polygon that encloses all points). This property provides a computationally cheap and effective way to approximate the curve's safety. By placing the control

points in a known free space, we have a strong guarantee that the resulting curve will also lie within that free space, simplifying collision checking.

- **Simplicity of Implementation:** For a single, C^1 continuous segment (which is sufficient for a turn), a cubic Bézier curve is mathematically straightforward and requires no complex setup like the knot vectors associated with splines.

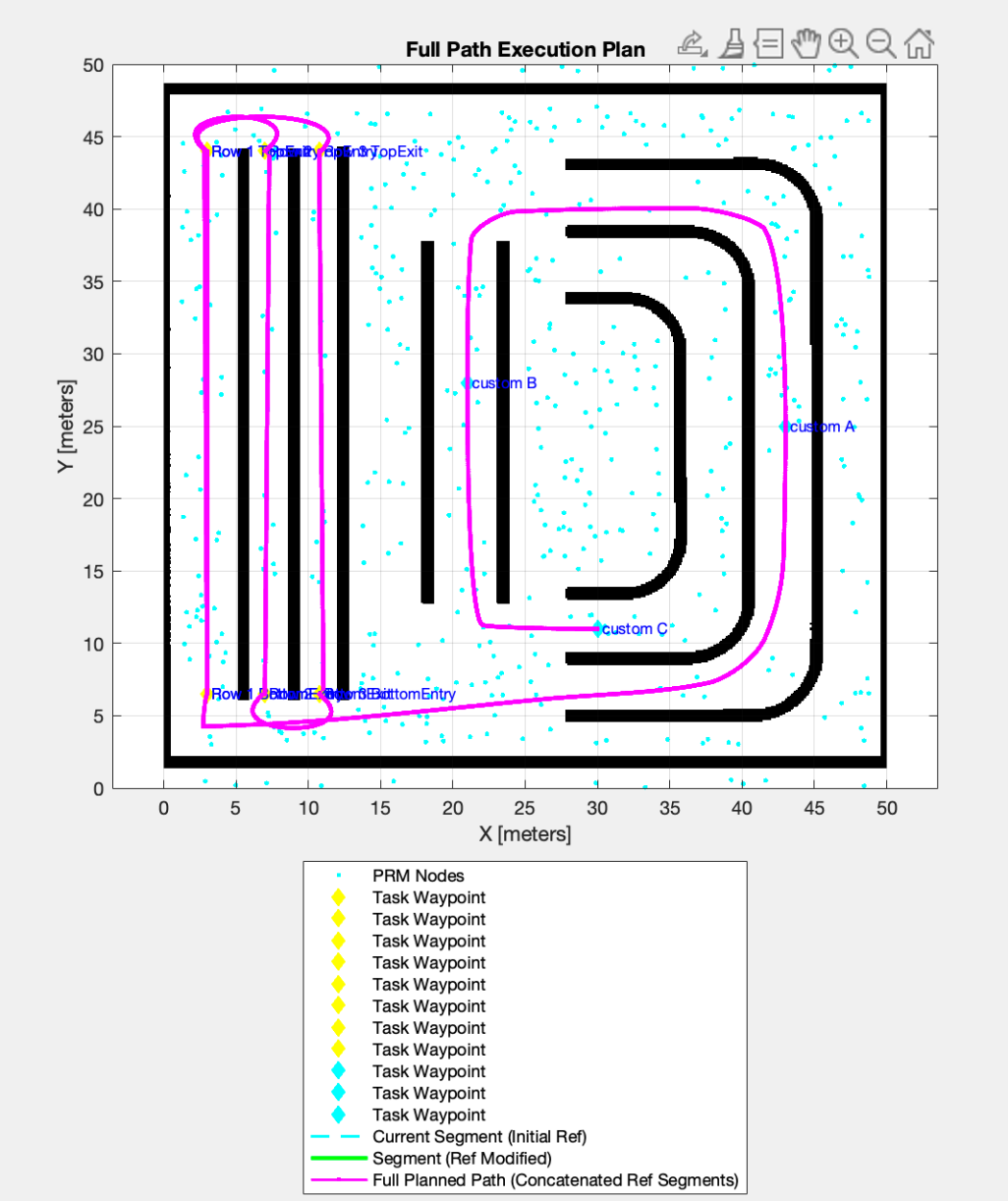
In contrast, splines offer local control, where moving a control point only affects a nearby portion of the curve. This is powerful for creating long, complex paths with varying curvature, but it introduces overhead (managing continuity between segments, knot placement) that is unnecessary for the single, self-contained turning maneuver required in the headland. Given the goal of creating a simple and robust heuristic, the global and predictable nature of Bézier curves was the more appropriate choice.



6. Results and Discussion

6.1. Mission Planning and Execution: The Snap-Point Logic

The developed system successfully plans and simulates complex missions. A key feature that enhances usability is the snap-point logic. Instead of requiring the user to specify exact coordinates, the mission is defined using high-level, semantic commands (e.g., Row_1_TopExit, Row_3_TopEntry). The application maintains a predefined dictionary of all significant locations in the field (row entries/exits, parking spots), each with its precise coordinates and required vehicle orientation. The planner automatically "snaps" the high-level commands to these detailed waypoints, building the mission sequence. This abstracts away the low-level geometric details, making mission definition highly intuitive and less prone to error. The final animation provides a clear visual validation of the entire process, showing the tractor-trailer correctly navigating the defined course.



6.2. Code Architecture and Modularity

The MATLAB code was structured to be modular and extensible, a critical aspect for facilitating future research. The architecture is centered around a main script that orchestrates the entire process, from mission definition to final simulation. This main script calls a series of specialized helper functions, each responsible for a specific task. Key modules include:

- **Parameter Definition:** Functions to define the vineyard geometry (defineVineyardParameters) and vehicle properties (geom).
- **Waypoint Management:** Functions to collect all predefined snap-points (collectAllSnapPoints) and to assemble the final mission sequence from user commands (getWaypointsFromMixedTask).
- **Path Generation:** A central function (generateSingleSegmentPath) that acts as a dispatcher, calling the appropriate method (straight line, Bézier, or PRM) based on the segment type.

- **Simulation and Control:** The core kinematic simulator (simulate_vehicle_from_pose) and the visualizer (animate_vehicles_with_map).
- **Path Modification:** The module responsible for analyzing and modifying paths (modify_curves_bezier), which was a central focus of the development.

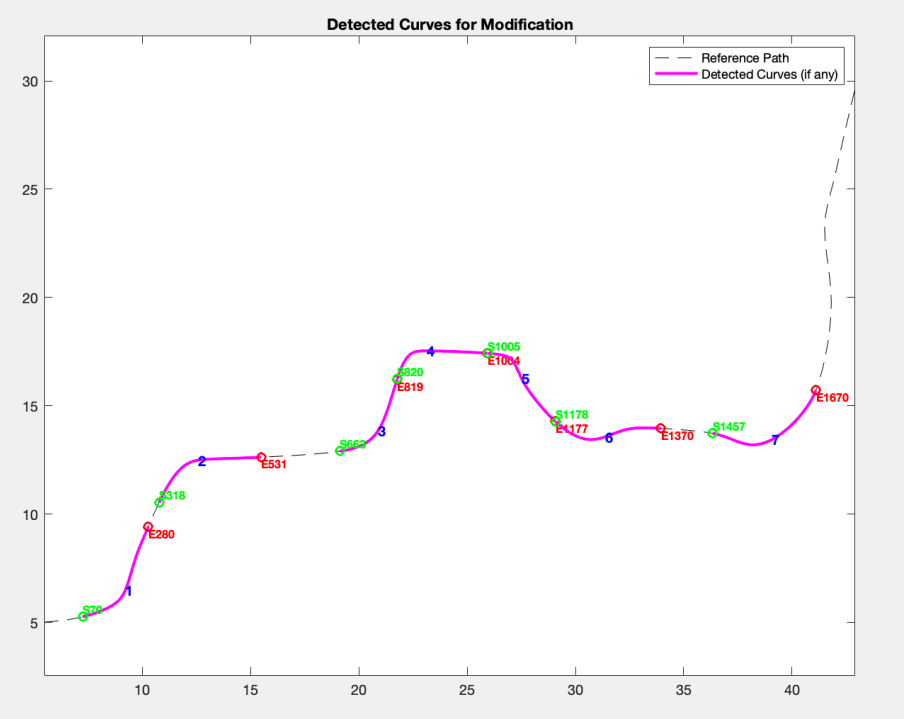
This modular design makes the code easier to understand, debug, and extend. For instance, adding a new type of controller would simply involve creating a new function and modifying the main loop to call it, without altering the path generation logic.

6.3. Performance of Headland Turn Generation

The heuristic-based Bézier curve generation for headland turns was shown to be a viable and efficient solution for the target scenario. It consistently produces smooth and safe turning paths without the computational overhead of an optimization-based approach. The visual results from the simulation confirm that the generated curves provide sufficient clearance for the trailer, effectively mitigating the off-tracking problem in this structured environment.

6.4. Preliminary Development for Open-Field Navigation

Beyond the structured vineyard environment, preliminary work was undertaken to generalize the path modification approach for open-field navigation. This involved the development of a simplistic curve detection module. The purpose of this module is to analyze a generic input path (e.g., one generated by a PRM planner in a cluttered environment) and automatically identify segments that are "curved." The detection is based on kinematic properties, using thresholds on the path's local curvature and the vehicle's simulated angular velocity. Once these curved segments are identified, the same Bézier modification logic used for headland turns could theoretically be applied to ensure their safety. This development is in its early stages but represents a crucial step towards creating a more versatile and robust path planner capable of operating in less structured environments.



6.5. Discussion of Limitations and Future Work

While the project was successful, it is important to acknowledge its limitations and the vast potential for future work that this environment enables. The current implementation relies on a known map and does not include dynamic obstacle avoidance. The heuristic for headland turns uses fixed parameters, which may not be optimal for different vehicle geometries or field layouts.

These limitations directly inform avenues for future research, which could be pursued in a subsequent thesis project, or future internships on this topic:

- **Advanced Control Strategies:** The current Pure Pursuit controller is effective but simple. Future work could explore more complex control methods like Model Predictive Control (MPC), a topic central to the Master's degree in Automation Engineering. This would allow for the explicit handling of constraints and optimization of control inputs.
- **Robustness and Odometry:** The current simulation and animation are coupled and use the same set of data. A valuable step would be to separate these processes to better focus on odometry. Introducing sensor noise (e.g., to wheel encoders, GPS) and implementing a state estimation filter (e.g., Kalman Filter) would create a more realistic simulation and test the robustness of the path tracking controller.
- **Advanced Maneuvers and Systems:** The current framework can be extended to tackle more complex problems. This includes developing a stable controller for **backwards motion**, a notoriously difficult task. Furthermore, the kinematic model could be generalized to handle N-trailer systems (vehicles with multiple trailers), introducing a new level of complexity to the planning problem.
- **Adaptive Heuristics and Open-Field Planning:** The fixed-parameter heuristic for Bézier curves could be made adaptive, tuning its parameters based on vehicle geometry and available space. This would go hand-in-hand with fully implementing and testing the curve detection and modification pipeline for robust open-field navigation.

7. Conclusions

This internship provided an invaluable, hands-on opportunity to dive into the complex and challenging domain of autonomous navigation for articulated vehicles. The experience has been crucial in solidifying and expanding my knowledge of autonomous driving, unmanned vehicles, and mobile robotics that began during my second year of bachelor's studies at Tongji University in a masterclass on the state of the art in the field.

The project demanded a comprehensive engagement with MATLAB, leading to a much more complete mastery of the language and its capabilities, particularly through the extensive use of the Robotics Toolbox™ and Navigation Toolbox™. Through the combination of literature review and practical software development, a deep, applied understanding of kinematic constraints, path planning algorithms, and control methodologies was achieved.

The final software application, while still being a first version, represents a concrete and successful outcome of the internship. It validates a hybrid, mission-decomposition approach to path planning and provides a robust and flexible environment for future experimentation.

The work has laid a solid foundation for future intenships in this same field, or eventually a thesis project, with clear directions for extending and improving the system's capabilities towards more robust and intelligent autonomous navigation for agricultural applications.

Finally, I would like to extend my sincere gratitude to the company, in particular to Nicolò Omodei and Simone Rossi for providing this stimulating challenge and for their guidance and throughout this learning experience.

8. References

[1] Recker, S., Krupke, D., & Lauer, M. (2021). "A Simple and Modular Approach to Path Planning for Tractor-Trailer Robots Based on Modification of Pre-Existing Trajectories." *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 6718-6724.

[2] Moreno, E. (2021). "Strategie di guida autonoma in retromarcia per robot mobili articolati." *Master's Thesis, Politecnico di Torino.*

[3] Beglini, M., Lanari, L., & Oriolo, G. (2020). "Anti-Jackknifing Control of Tractor-Trailer Vehicles via Intrinsically Stable MPC." *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8806-8812.

[4] Cariou, C., Lenain, R., Thuilot, B., Humbert, T., & Berducat, M. (2010). "Maneuvers automation for agricultural vehicle in headland." *AgEng 2010, International Conference on Agricultural Engineering.*

[5] Jiangyi, H., Hengyuan, L., & Hao, T. (2025). "Research on Obstacle Avoidance Path Planning for Wheeled Tractor-Trailer System." *International Journal of Automotive Technology*, 26, pp. 1067-1076.

[6] Hao, Y., & Agrawal, S. K. (2005). "Formation Planning and Control of UGVs with Trailers." *Autonomous Robots*, 19, pp. 257-270.

[7] Coulter, R. C. (1992). "Implementation of the Pure Pursuit Path Tracking Algorithm." *Carnegie Mellon University, Robotics Institute, Technical Report CMU-RI-TR-92-01.*

[8] Dubins, L. E. (1957). "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents." *American Journal of Mathematics*, 79(3), pp. 497-516.

[9] Reeds, J., & Shepp, L. (1990). "Optimal paths for a car that goes both forwards and backwards." *Pacific Journal of Mathematics*, 145(2), pp. 367-393.

[10] https://it.mathworks.com/ “Matlab Documentation”